

Jenkins' architecture

Jenkins, as a standalone server operating on a single machine, can deliver services to a certain extent only. The quality of service it renders depends on the resources of the machine it is running on.

However, some projects require different platforms and environments to build certain stages. Also, to test those different builds, different environments are essential. Let us look at some examples of such environments:

- One project requires JDK 1.7 but the other project requires JDK 1.8.
- Two different Android projects require distinct versions of a build tool (e.g., Gradle 4 and Gradle 4.6).
- Similarly, several iOS projects might depend on different XCode versions.

Likewise, when a platform is considered, iOS development projects have a dependency on iOS machines, Windows environment on .NET, and some might require the Linux environment.

Thus, a single machine cannot fulfil all the pre-requisites and dependencies required to build and test the project. A single machine also cannot give efficient service if the project is very large, and requires building and testing on a very frequent basis. Considering these scenarios, it can be concluded that a single Jenkins server is incapable of catering to the diverse needs of projects, business units or organisations.

To deal with the aforesaid limitations, Jenkins Distributed Architecture, which follows the master-agent architecture, has now become an integral part of the Jenkins architecture. The master is where the main Jenkins server is running and the agent is the one which runs on a remote machine. There can be only one master but many agents. There are many ways in which an agent can be connected to the master but, ultimately, they need to have bi-directional communication, and for this the TCP/IP protocol suite is used. Figure 1 depicts the Jenkins

Table 2: Features of containers

Features	Description
Flexible	Complex applications can also be containerised
Lightweight	Unlike virtual machines, containers run directly within the host machine's kernel instead of running on top of a hypervisor
Portable	Containers can be built and configured locally, shared as an image, be deployed on other systems and platforms (even the cloud), and run from anywhere
Scalable	On deployment, users can define how many replicas of the containers need to be run. They are as easy to scale down, as they are to scale up
Interchangeable	Updates can be deployed in runtime
Stackable	In runtime, services can be stacked vertically

Table 3: A description of Docker components

Component	Description
Docker engine	Docker engine is a client-server application that is responsible for creating, running and managing the containers. This core is made up of the three components depicted in Figure 2: • Docker daemon is a server that constantly runs on the host. • REST API guides the programs to interact with the Docker daemon. • The command line interface is to execute Docker commands.
Docker daemon	Docker daemon executes the issues submitted by the Docker API. It administers and maintains the Docker images, containers, volumes and networks. It is also accountable to manage communications with other such daemons in order to manage the Docker services.
Docker client	Docker client is the user end, which takes the user input and conveys it to the Docker daemon. It is capable of communicating with multiple such Docker daemons.
Dockerfile	The instructions to build a Docker image are written in an extensionless text file known as Dockerfile. Each instruction when processed means a layer has been added to the Docker image.

```

ajay@lenovo-g50-80: ~
File Edit View Search Terminal Help
ajay@lenovo-g50-80:~$ docker --version
Docker version 18.09.5, build e8ff056
ajay@lenovo-g50-80:~$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
ajay@lenovo-g50-80:~$

```

Figure 3: Running Docker commands

```

ajay@lenovo-g50-80: ~
File Edit View Search Terminal Help
ajay@lenovo-g50-80:~$ docker --version
Docker version 18.09.5, build e8ff056
ajay@lenovo-g50-80:~$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
ajay@lenovo-g50-80:~$ docker run -u root -d -p 8080:8080 -p 50000:50000 -v /home/ajay/Desktop/jenkins_home:/var/jenkins_home/ -v /var/run/docker.sock:/var/run/docker.sock --restart always jenkinsct/blueocean
0992445f7d2a378178d4d29c8238eb8f3a1a44aba98de10ba4b721aabb5a578a
ajay@lenovo-g50-80:~$

```

Figure 4: Running a container

master-agent architecture, where there is one master running on a MacOS system, and the three agents connected with it run on different platforms like MacOS, Windows and Linux. For better performance and optimisation,

it is considered best if the master node is only used for monitoring and distributing the build jobs among the agents, and doesn't perform any build on itself. With this, the agents can provide a variety of platforms and